Faculty of Engineering
Electronics and Electrical Communication Department

ELC 3080

TCP & OSPF Lab

Name	Section	ID	BN
رنا محمد محمود محمد علي	2	9221266	12
سهيله احمد محمد عبدالمجيد	2	9220377	23

Under the supervision of:

Dr/ Rania Osama

TA/ Ahmed Atef

Table of Contents

1. TCP Lab 3

1.1. Effect of TCP Window Size 3

1.2. TCP short versus long paths 7

1.3. Higher Link Capacity with Drops versus Reliable Lower Capacity 7

2. OSPF Lab 9

2.1. OSPF Link Cost Change 9

2.2. OSPF Database Updates 12

1. TCP Lab

1.1. Effect of TCP Window Size

i. Vary iperf3 window size from 1Kbytes to 6 Kbytes in increments of 1 Kbyte, then set it to 12, 16, 24, 32 Kbytes. Plot the avg throughput and the avg number of retransmissions as function of window size. Comment on the results and explain the zigzag behaviour noticed for larger window sizes. Note that number of retransmissions is the 5th column in iperf3 default output.

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 1k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40846 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec  2.52 MBytes  2.12 Mbits/sec    0   5.45 KBytes
[  4]  10.00-20.00  sec  1.95 MBytes  1.64 Mbits/sec    0   5.45 KBytes
[  4]  20.00-30.00  sec  2.71 MBytes  2.27 Mbits/sec    0   5.45 KBytes
[  4]  30.00-40.00  sec  2.43 MBytes  2.04 Mbits/sec    0   5.45 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec  9.61 MBytes  2.02 Mbits/sec    0             sender
[  4]   0.00-40.00  sec  9.61 MBytes  2.02 Mbits/sec                  receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 1: Window Size = 1 KB

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 2k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40851 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec  6.07 MBytes  5.09 Mbits/sec    0   7.07 KBytes
[  4]  10.00-20.00  sec  7.33 MBytes  6.15 Mbits/sec    0   7.07 KBytes
[  4]  20.00-30.00  sec  5.59 MBytes  4.69 Mbits/sec    0   7.07 KBytes
[  4]  30.00-40.00  sec  6.10 MBytes  5.12 Mbits/sec    0   7.07 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec  25.1 MBytes  5.26 Mbits/sec    0             sender
[  4]   0.00-40.00  sec  25.1 MBytes  5.26 Mbits/sec                  receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 1: Window Size = 2 KB

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 3k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40871 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec  10.5 MBytes  8.79 Mbits/sec    0   14.1 KBytes
[  4]  10.00-20.00  sec  9.79 MBytes  8.21 Mbits/sec    0   14.1 KBytes
[  4]  20.00-30.00  sec  8.69 MBytes  7.29 Mbits/sec    0   14.1 KBytes
[  4]  30.00-40.00  sec  9.96 MBytes  8.35 Mbits/sec    0   14.1 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec  38.9 MBytes  8.16 Mbits/sec    0             sender
[  4]   0.00-40.00  sec  38.9 MBytes  8.16 Mbits/sec                  receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 3: Window Size = 3 KB

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40875 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec  9.60 MBytes  8.06 Mbits/sec    0   14.1 KBytes
[  4]  10.00-20.00  sec  8.54 MBytes  7.17 Mbits/sec    0   14.1 KBytes
[  4]  20.00-30.00  sec  9.80 MBytes  8.22 Mbits/sec    0   14.1 KBytes
[  4]  30.00-40.00  sec  9.50 MBytes  7.97 Mbits/sec    0   14.1 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec  37.5 MBytes  7.85 Mbits/sec    0             sender
[  4]   0.00-40.00  sec  37.4 MBytes  7.85 Mbits/sec                  receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 4: Window Size = 4 KB

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 5k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40879 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec  9.91 MBytes  8.31 Mbits/sec    0   14.1 KBytes
[  4]  10.00-20.00  sec  9.55 MBytes  8.01 Mbits/sec    0   14.1 KBytes
[  4]  20.00-30.00  sec  10.1 MBytes  8.45 Mbits/sec    0   14.1 KBytes
[  4]  30.00-40.00  sec  9.93 MBytes  8.33 Mbits/sec    0   14.1 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec  39.5 MBytes  8.28 Mbits/sec    0             sender
[  4]   0.00-40.00  sec  39.5 MBytes  8.28 Mbits/sec                  receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 5: Window Size = 5 KB

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 6k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40887 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec   154 KBytes   126 Kbits/sec   88   4.24 KBytes
[  4]  10.00-20.00  sec   148 KBytes   122 Kbits/sec   89   4.24 KBytes
[  4]  20.00-30.00  sec   148 KBytes   122 Kbits/sec   89   4.24 KBytes
[  4]  30.00-40.00  sec   143 KBytes   117 Kbits/sec   88   4.24 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec   594 KBytes   122 Kbits/sec  354             sender
[  4]   0.00-40.00  sec   584 KBytes   120 Kbits/sec                  receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 6: Window Size = 6 KB

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 12k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40892 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec  3.70 MBytes  3.11 Mbits/sec  667   4.24 KBytes
[  4]  10.00-20.00  sec  3.35 MBytes  2.81 Mbits/sec  606   4.24 KBytes
[  4]  20.00-30.00  sec  3.10 MBytes  2.60 Mbits/sec  563   2.83 KBytes
[  4]  30.00-40.00  sec  3.94 MBytes  3.31 Mbits/sec  713   4.24 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec  14.1 MBytes  2.96 Mbits/sec  2549             sender
[  4]   0.00-40.00  sec  14.1 MBytes  2.95 Mbits/sec                   receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 7: Window Size = 12 KB

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 16k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40908 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec   403 KBytes   330 Kbits/sec  146   4.24 KBytes
[  4]  10.00-20.00  sec   382 KBytes   313 Kbits/sec  127   7.07 KBytes
[  4]  20.00-30.00  sec   438 KBytes   359 Kbits/sec  130   7.07 KBytes
[  4]  30.00-40.00  sec   445 KBytes   365 Kbits/sec  132   7.07 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec  1.63 MBytes   342 Kbits/sec  535              sender
[  4]   0.00-40.00  sec  1.61 MBytes   337 Kbits/sec                   receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 8: Window Size = 16 KB

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 24k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40910 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec   329 KBytes   270 Kbits/sec  121   7.07 KBytes
[  4]  10.00-20.00  sec   297 KBytes   243 Kbits/sec  119   5.66 KBytes
[  4]  20.00-30.00  sec   297 KBytes   243 Kbits/sec  120   5.66 KBytes
[  4]  30.00-40.00  sec   297 KBytes   243 Kbits/sec  120   5.66 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec  1.19 MBytes   250 Kbits/sec  480              sender
[  4]   0.00-40.00  sec  1.15 MBytes   241 Kbits/sec                   receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 9: Window Size = 24 KB

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 32k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40912 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec   274 KBytes   225 Kbits/sec  111   5.66 KBytes
[  4]  10.00-20.00  sec   161 KBytes   132 Kbits/sec   94   18.4 KBytes
[  4]  20.00-30.00  sec   161 KBytes   132 Kbits/sec   94   29.7 KBytes
[  4]  30.00-40.00  sec   242 KBytes   198 Kbits/sec  118   14.1 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec   839 KBytes   172 Kbits/sec  417              sender
[  4]   0.00-40.00  sec   788 KBytes   161 Kbits/sec                   receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 10: Window Size = 32 KB

Window Size	Throughput	Retransmissions	Window Size	Throughput	Retransmissions
1 KB	2.02 MBPS	0	6 KB	122 KBPS	354
2 KB	5.26 MBPS	0	12 KB	2.96 MBPS	2549
3 KB	8.16 MBPS	0	16 KB	342 KBPS	535
4 KB	7.85 MBPS	0	24 KB	250 KBPS	480
5 KB	8.28 MBPS	0	32 KB	172 KBPS	417

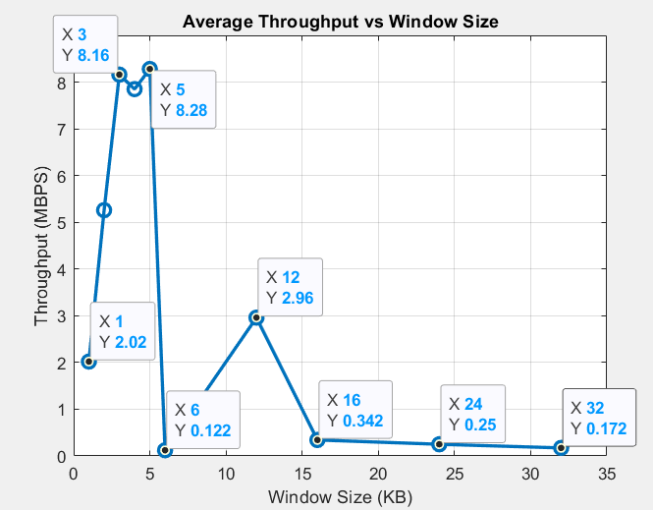


Figure 11: Throughput (MBPS) vs Window Size (KB)

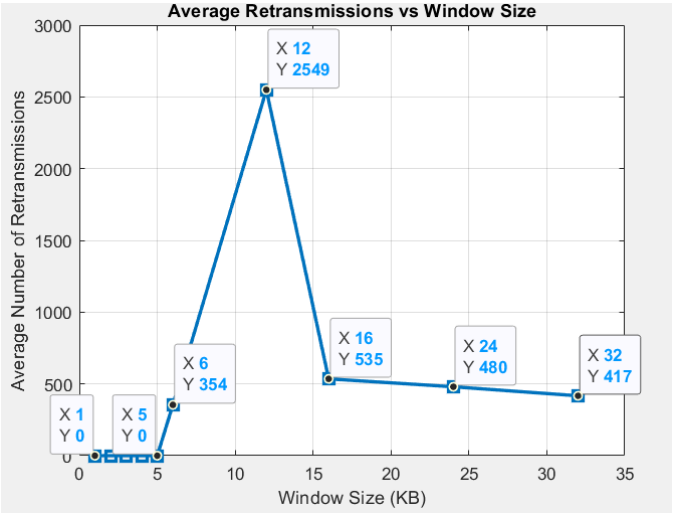


Figure 12: Average Retransmissions vs Window Size (KB)

Comment

From the throughput versus window size graph, we observe a zigzag pattern as the window size increases from 1 KB to 5 KB. In this range, the number of retransmissions remains zero, indicating that the receiver's buffer is not full yet. As a result, throughput increases steadily with window size. At 5 KB, the receiver buffer reaches its limit, and at 6 KB, a sharp drop in throughput occurs with an increase in the number of retransmissions due to packet loss. This triggers the sender to adjust its transmission strategy, allocating more buffer space or slowing down the sending rate. This leads to an increase in throughput between 6 KB and 12 KB, but the retransmissions remain high. Then, when the window size reach 16 KB, the receiver buffer becomes full again, leading to more packet loss, a decrease in throughput, and the number of retransmissions.

ii. Click on any of the TCP a data segment whose source is node n7, dissect the segment by following different protocol layer headers from TCP->IP->Ethernet identifying how many header bytes are added by each layer, identify the TCP options used.

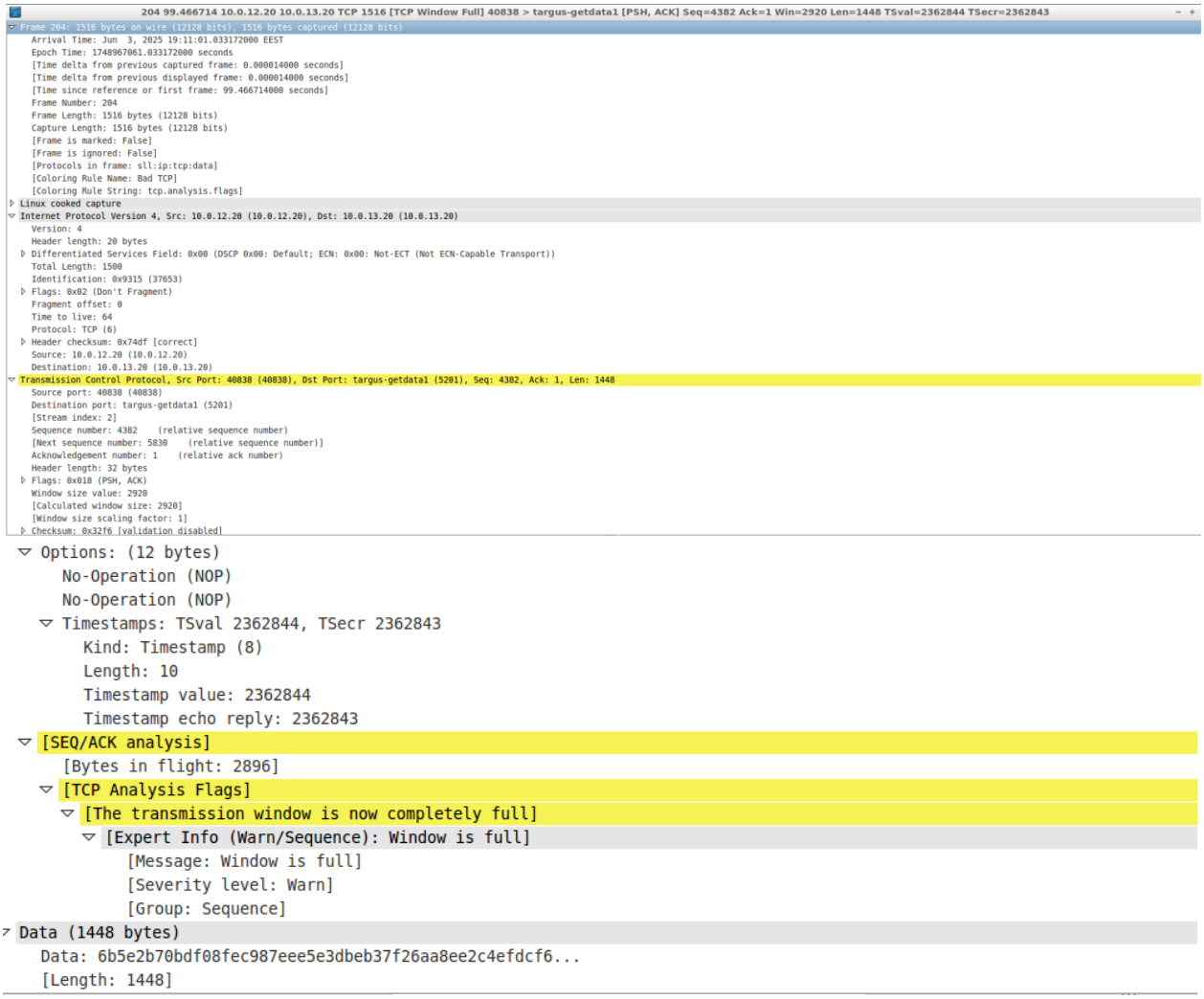


Figure 13: TCP Data Packet (sent from n7 to n11)

Frame Length	1516 bytes
TCP Header	32 bytes
IP Header	20 bytes
Ethernet Header	Frame Length – TCP Header – IP Header = 1516 – 32 – 20 = 1464 bytes
TCP Options	12 bytes (2 NOP + 10 Timestamp)
TCP Options (Timestamp)	10 bytes

iii. Repeat for an ACK packet sent from node n11.

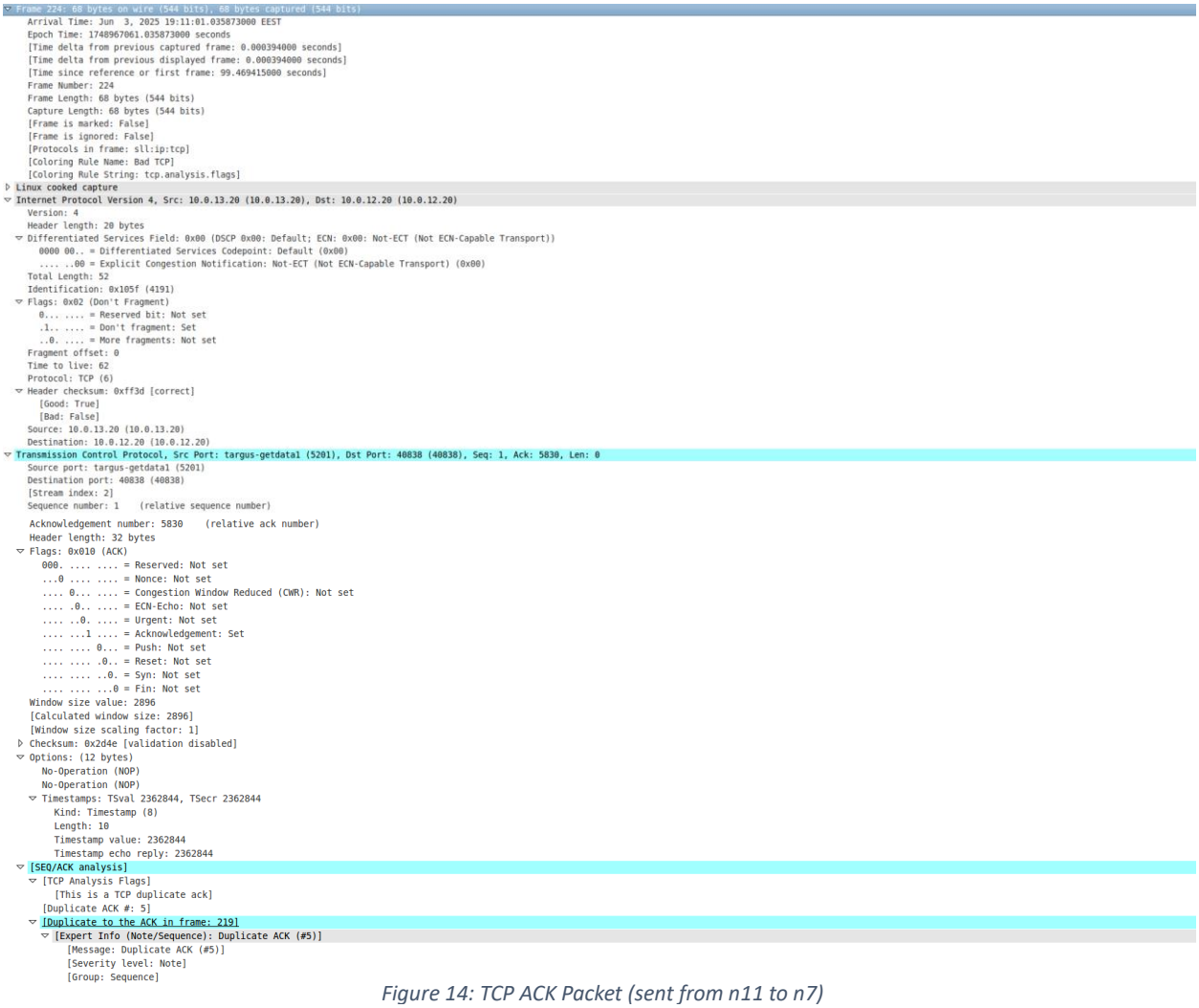


Figure 14: TCP ACK Packet (sent from n11 to n7)

Frame Length	68 bytes
TCP Header	32 bytes
IP Header	20 bytes
Ethernet Header	Frame Length – TCP Header – IP Header = 68 – 32 – 20 = 16 bytes
TCP Options	12 bytes (2 NOP + 10 Timestamp)
TCP Options (Timestamp)	10 bytes

1.2. TCP short versus long paths

i. Compare the result of throughput with the case when connection was made to node n11. Why throughput drops when connecting to n8 although capacities on the two paths are the same.

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.13.20, port 5201
[  4] local 10.0.12.20 port 40875 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec  9.60 MBytes  8.06 Mbits/sec    0   14.1 KBytes
[  4]  10.00-20.00  sec  8.54 MBytes  7.17 Mbits/sec    0   14.1 KBytes
[  4]  20.00-30.00  sec  9.80 MBytes  8.22 Mbits/sec    0   14.1 KBytes
[  4]  30.00-40.00  sec  9.50 MBytes  7.97 Mbits/sec    0   14.1 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec  37.5 MBytes  7.85 Mbits/sec    0             sender
[  4]   0.00-40.00  sec  37.4 MBytes  7.85 Mbits/sec                  receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 15: Short Path (from n7 to n11)

```
root@n7:/tmp/pycore.51726/n7.conf# iperf3 -c 10.0.11.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.11.20, port 5201
[  4] local 10.0.12.20 port 37577 connected to 10.0.11.20 port 5201
[ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
[  4]   0.00-10.00  sec  8.01 MBytes  6.72 Mbits/sec    0   14.1 KBytes
[  4]  10.00-20.00  sec  7.61 MBytes  6.38 Mbits/sec    0   14.1 KBytes
[  4]  20.00-30.00  sec  7.20 MBytes  6.04 Mbits/sec    0   14.1 KBytes
[  4]  30.00-40.00  sec  8.05 MBytes  6.75 Mbits/sec    0   14.1 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-40.00  sec  30.9 MBytes  6.47 Mbits/sec    0             sender
[  4]   0.00-40.00  sec  30.9 MBytes  6.47 Mbits/sec                  receiver

iperf Done.
root@n7:/tmp/pycore.51726/n7.conf#
```

Figure 16: Long Path (from n8 to n7)

Comment

- Throughput from n7 to n11 (short path) = 7.85 MBPS.
- Throughput from n7 to n8 (long path) = 6.47 MBPS.
- As observed, shorter paths result in lower delays, which lead to higher throughput (n7 to n11). In contrast, longer paths introduce greater delays, resulting in reduced throughput(n7 to n8).

1.3. Higher Link Capacity with Drops versus Reliable Lower Capacity

a. Select link between n4 and n5, configure it to have capacity of 10 Mbps with zero loss in both directions.

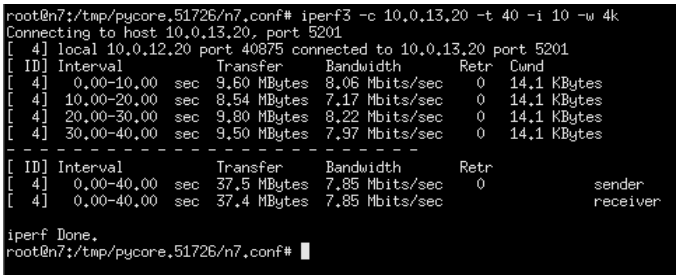
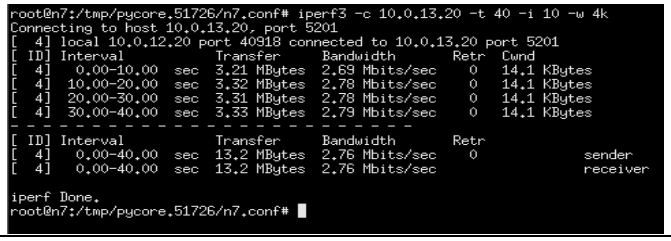
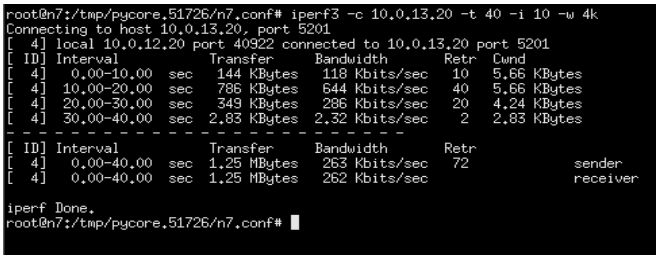
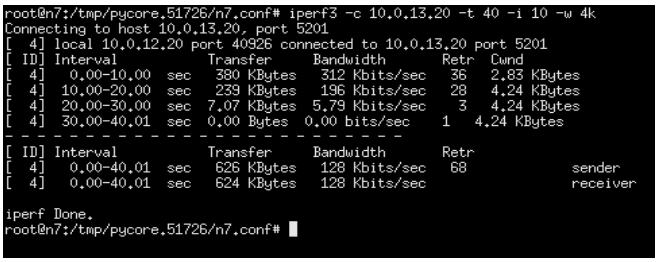
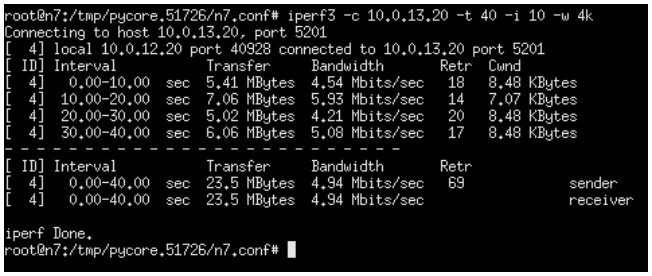
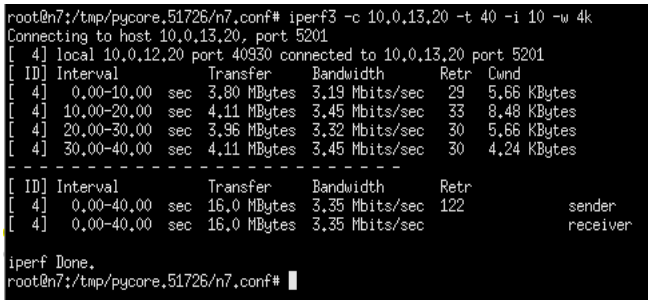
b. Select link between n4 and n5, configure it to have capacity of 3 Mbps with zero loss in both directions.

c. Select link between n4 and n5, configure it to have capacity of 10 Mbps with 5% loss in both directions.

d. Select link between n4 and n5, configure it to have capacity of 100 Mbps with 10% loss in both directions.

e. Select link between n4 and n5, configure it to have capacity of 10 Mbps with 1% loss in direction from n4 to n5 and 0% loss in the other direction.

f. Select link between n4 and n5, configure it to have capacity of 10 Mbps with 0% loss in direction from n4 to n5 and 1% loss in the other direction.

Case	Throughput	Figure
a	7.85 MBPS	
b	2.76 MBPS	
c	0.263 MBPS	
d	0.128 MBPS	
e	4.94 MBPS	
f	3.35 MBPS	

i. Compare throughputs in cases a, b, c. Why b is better than c?

We notice that throughput in case (a) is greater than that in case (b) as the link capacity in case (a) is higher. We notice that throughput in case (a) is greater than that in case (c) as the link in case (a) has no loss unlike case (c) (5%). We also notice that that throughput in case (b) is greater than that in case (c) as there is a higher loss in case (c) (5%) than that in case (b) (no loss) although the link capacity in case (b) is smaller. Which means (b) has a better performance than (c).

ii. Compare throughputs in cases b, c and d. Which is better? Why?

Case (b) is better than case (c) and (d) (throughput in case (b) is greater than that in case (c) and (d)). This happens as there is no loss in case (b) compared to case (c) (5%) and (d) (10%) which means there are no retransmissions needed in case (b) although it has smaller link capacity than (c) and (d). this leads to a better performance in case (b) compared to the other 2 cases.

iii. Compare throughputs in cases e and f? Which is better? Why?

Throughput in case (e) is greater than that in case (f), which means case (e) is better. This happens as the data sent from n5 to n4 (from client to server) is greater than the data sent from n4 to n5 (from server to client), so when the loss is greater in the upstream link (from client to server) the performance becomes worse as the number of the lost packets increases (as we can see in case (f)).

2. OSPF Lab

Each of the following sections describes an experiment based on the network topology provided by the Project_OSPF_TCP.imn file. The Introductory Lab explained and introduced most of the needed tools like iperf3, vtysh, wireshark, etc. If you have not gone through it, do not start working on the assignment below. It is better to start the network emulation once and run Wireshark captures. You can reset Wireshark captures from experiment to another.

2.1. OSPF Link Cost Change

1. Stop any running iperf3 clients.
2. Set all links to have zero loss in the two directions with 100 Mbps speed.
3. Run iperf3 between n7 and n11 for a duration of 500 seconds or longer. Identify the path between n7 and n11.
4. Open vtysh on node n5 by opening a bash terminal and typing vtysh. Now we can configure the router and link costs.
5. Type the following in vtysh: `show Ip ospf interface eth1` (To know the ethernet number of each interface you can choose toolbar->view->show->interface names) This displays information about interface eth1. Note its ospf cost.
6. Type the following configure terminal.

- `interface eth1` (the interface for the link between n5 and n4)
- `ospf cost 40` (set cost to 40)

2.1.1. Check what happens to the path between n7 and n11 (as seen after steps 3 and 6)?Explain what happens.

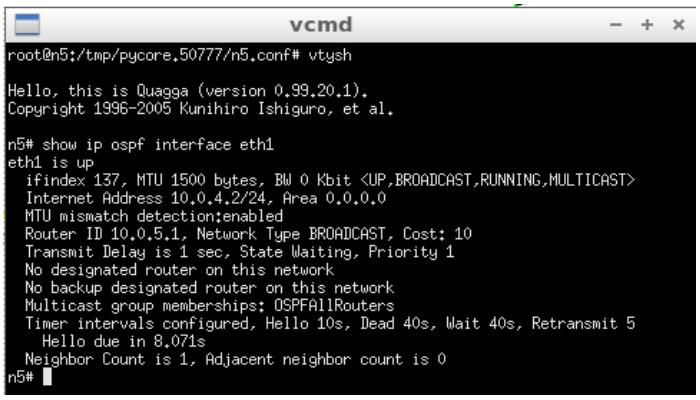


Figure 17 n5 interface 1 before changing cost

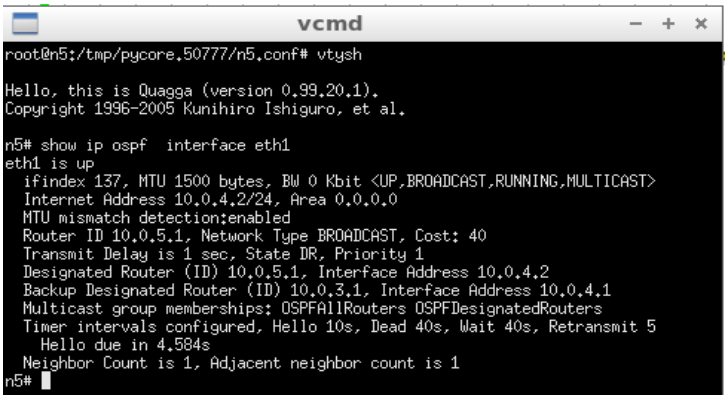


Figure 18 n5 interface 1 after changing cost

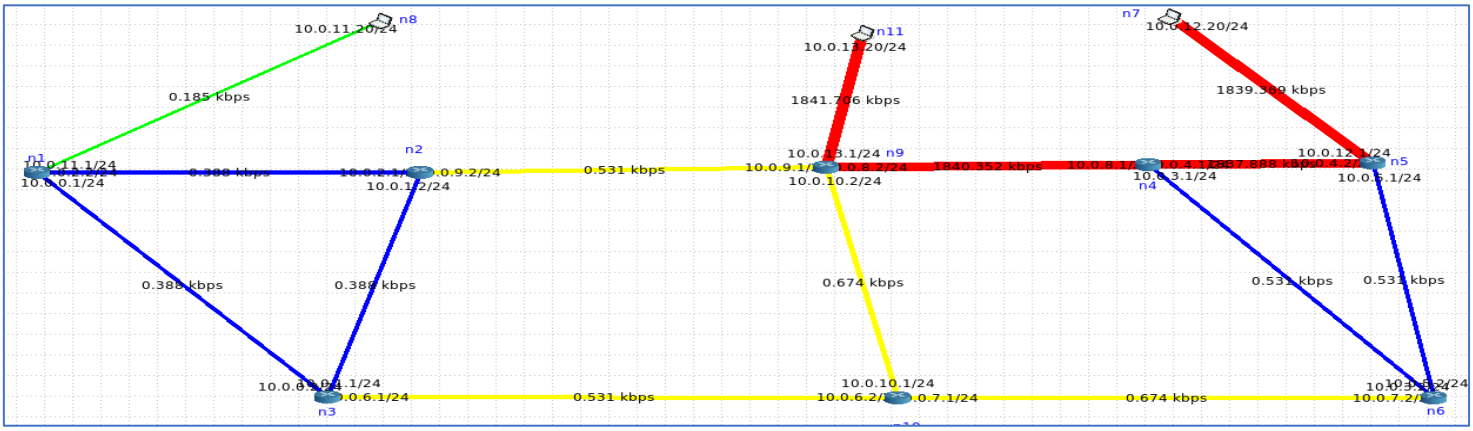


Figure 19 Path From n7 to n11 before changing n5 cost.

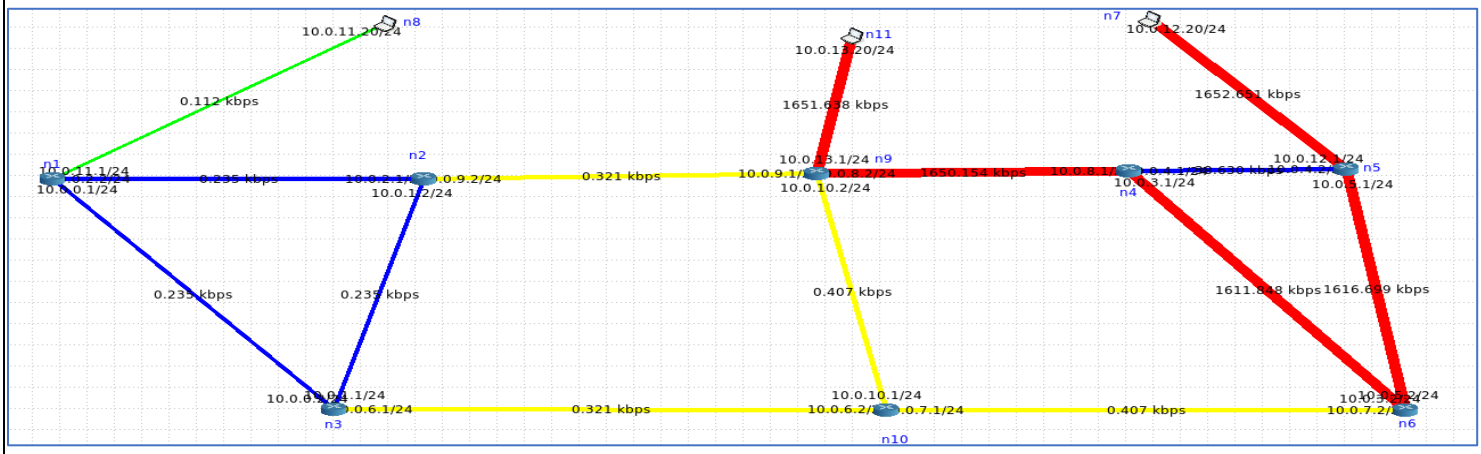


Figure 30 Path From n7 to n11 after changing n5 cost.

Comment on your findings:

Increasing the cost of the interface between n4 and n5 causes the data packets to take an alternative route to achieve a lower overall path cost, with the help of Dijkstra's Algorithm. The routers begin to recalculate the best route to determine the shortest path between n7 and n11.

2.1.2. Set the cost of eth1 at node n5 back to 10. Establish two iperf3 connections: one from n7 to n11 and the second from n11 to n7 both for duration of 500 seconds. Now go to node n4 and set interface cost for interface connecting n4 with n5 to 40. What happens in the paths of the two connections? Explain what happens. What do you conclude?

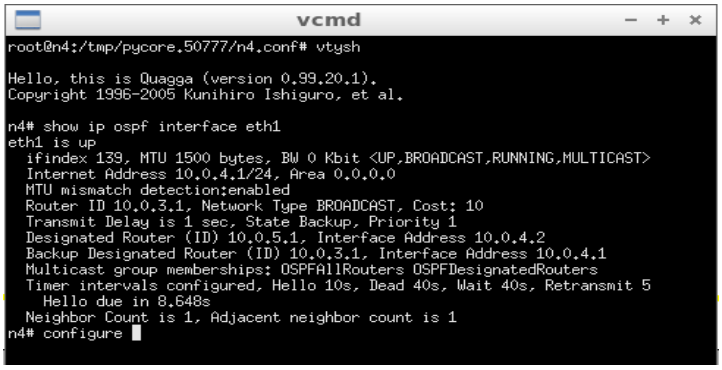


Figure 21 n4 interface 1 before changing cost

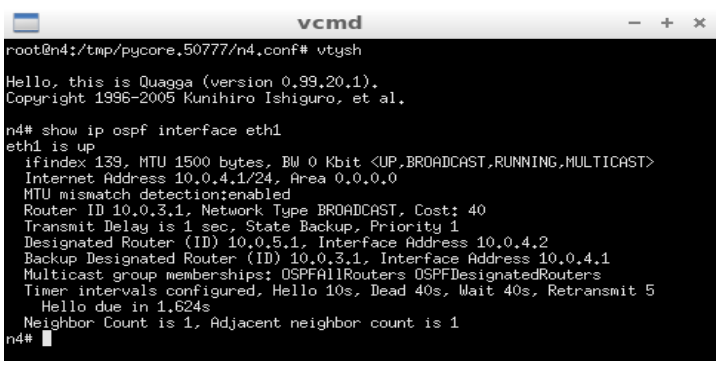


Figure 22 n4 interface 1 AFTER changing cost

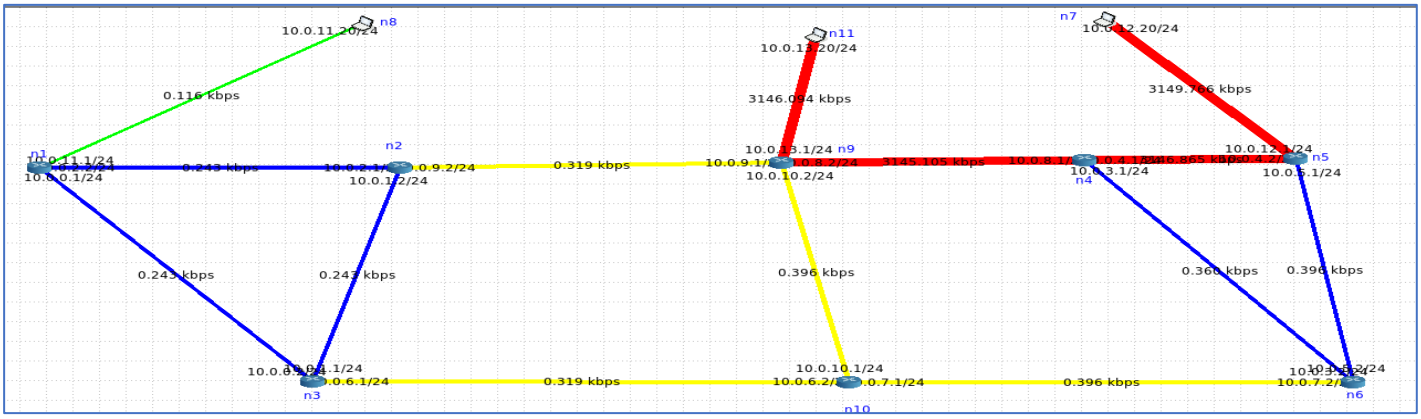


Figure 23 Path From n7 to n11 before changing n4 cost.

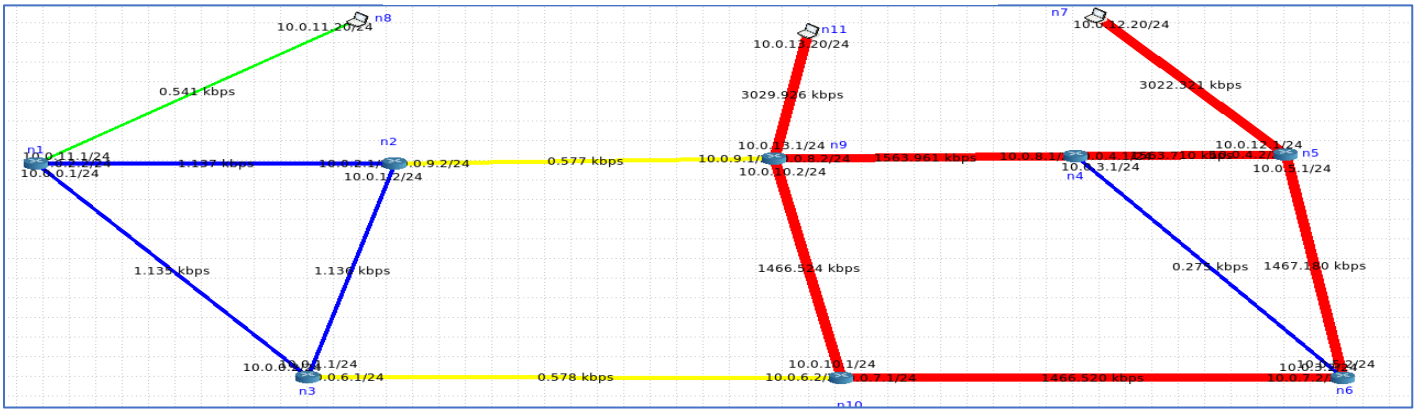


Figure 24 Path From n7 to n11 after changing n4 cost.

Comment on your findings:

Increasing cost of a link forces the router to choose an alternative path, even if it is longer, to reduce congestion. As OSPF could handle such situations.by increasing the cost of n4 → n5 and leaving n5 → n4 the same. The shortest path from n7 to n11 is not the same as the shortest path from n11 to n7. Paths are (n7 → n5 → n4 → n9 → n11) & (n11 → n9 → n10 →n6→ n5→n7)

2.2. OSPF Database Updates

1. Start wireshark and have its filter to capture only OSPF related packets.
2. Go to another router say router n2, open vtysh and issue the commands:

- `show ip ospf database` (check this link to understand more)
- `show ip ospf route.`

3. On router n4, set link cost of eth1 to 20. Capture the link state packets advertised in Wireshark.
4. Go to router n4, go out of vtysh, or open new bash terminal and issue the following commands to disconnect router n4 from the network:

- `ifconfig eth0 down` - `ifconfig eth1 down` - `ifconfig eth2 down`

2.2.1. Capture and explain the outputs due to execution of step 2. Why some destinations have more than route in the routing tables?

```
Hello, this is Quagga (version 0.99.20.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

n2# show ip ospf database

       OSPF Router with ID (10.0.1.2)

                Router Link States (Area 0.0.0.0)

Link ID         ADV Router      Age  Seq#       CkSum  Link count
10.0.0.1        10.0.0.1        1362 0x8000000a 0xf2bd 3
10.0.0.2        10.0.0.2        1360 0x8000000a 0x9a08 3
10.0.1.2        10.0.1.2        1357 0x8000000a 0x8d0a 3
10.0.3.1        10.0.3.1         845 0x8000000d 0x412d 3
10.0.3.2        10.0.3.2        1360 0x8000000a 0xadde 3
10.0.5.1        10.0.5.1        1144 0x8000000a 0x9007 3
10.0.6.2        10.0.6.2        1357 0x80000009 0xccab 3
10.0.8.2        10.0.8.2        1356 0x8000000b 0x2414 4

                Net Link States (Area 0.0.0.0)

Link ID         ADV Router      Age  Seq#       CkSum
10.0.0.2        10.0.0.2        1364 0x80000001 0x5fcb
10.0.1.2        10.0.1.2        1357 0x80000001 0x64c2
10.0.2.1        10.0.1.2        1362 0x80000001 0x51d6
10.0.3.2        10.0.3.2        1364 0x80000001 0x6bb3
10.0.4.2        10.0.5.1         244 0x80000002 0x62b8
10.0.5.1        10.0.5.1         165 0x80000002 0x67b2
10.0.6.2        10.0.6.2        1364 0x80000001 0x4bcc
10.0.7.1        10.0.6.2        1365 0x80000001 0x59bb
10.0.8.2        10.0.8.2         162 0x80000002 0x50be
10.0.9.1        10.0.8.2        1362 0x80000001 0x2de3
10.0.10.2       10.0.8.2        1357 0x80000001 0x61a8
```

Figure 25 n2 Database before

```
n2# show ip ospf route
============ OSPF network routing table ============
N    10.0.0.0/24           [20] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.2.2, eth1
N    10.0.1.0/24           [10] area: 0.0.0.0
                           directly attached to eth0
N    10.0.2.0/24           [10] area: 0.0.0.0
                           directly attached to eth1
N    10.0.3.0/24           [30] area: 0.0.0.0
                           via 10.0.9.1, eth2
N    10.0.4.0/24           [50] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.9.1, eth2
N    10.0.5.0/24           [40] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.9.1, eth2
N    10.0.6.0/24           [20] area: 0.0.0.0
                           via 10.0.1.1, eth0
N    10.0.7.0/24           [30] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.9.1, eth2
N    10.0.8.0/24           [20] area: 0.0.0.0
                           via 10.0.9.1, eth2
N    10.0.9.0/24           [10] area: 0.0.0.0
                           directly attached to eth2
N    10.0.10.0/24          [20] area: 0.0.0.0
                           via 10.0.9.1, eth2
N    10.0.11.0/24          [20] area: 0.0.0.0
                           via 10.0.2.2, eth1
N    10.0.12.0/24          [50] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.9.1, eth2
N    10.0.13.0/24          [20] area: 0.0.0.0
                           via 10.0.9.1, eth2
```

Figure 26 n2 Routing table before

Comment on your findings:

Several routes share the same cost, letting the switch pick any for forwarding. If one fails, others ensure connectivity.

Router n2's database has two parts:

1. **Router Link State**: Complete network topology learned via hello exchanges.
2. **Designated Router Link State**: Summarized topology for a multi-access segment, advertised by the elected designated router to reduce flooding.

Each LSA entry shows:

- **Link ID** (network or router described)
- **ADV Router** (originator)
- **Age** (time since last refresh)
- **SEQ#** (to identify newer LSAs)
- **Checksum** (packet integrity check)
- **Link Count** (number of links in the LSA)

The routing table lists destinations and their next hops. OSPF's ECMP allows multiple equal-cost paths to the same destination, so some entries list several equivalent routes, while others have just one.

2.2.2. After executing step 3, determine how long it took the network to exchange link state packets and adjust routing tables. (Hint: you can calculate the required time by observing the time of first OSPF update message and the last ACK from Wireshark).

```
n4# show ip ospf interface eth1
eth1 is up
  ifindex 139, MTU 1500 bytes, BW 0 Kbit <UP,BROADCAST,RUNNING,MULTICAS
  Internet Address 10.0.4.1/24, Area 0.0.0.0
  MTU mismatch detection:enabled
  Router ID 10.0.3.1, Network Type BROADCAST, Cost: 20
  Transmit Delay is 1 sec, State Backup, Priority 1
  Designated Router (ID) 10.0.5.1, Interface Address 10.0.4.2
  Backup Designated Router (ID) 10.0.3.1, Interface Address 10.0.4.1
  Multicast group memberships: OSPFAllRouters OSPFDesignatedRouters
  Timer intervals configured, Hello 10s, Dead 40s, Wait 40s, Retransmit
    Hello due in 2.491s
  Neighbor Count is 1, Adjacent neighbor count is 1
n4#
```

Figure 27 n4 interface 1 after changing cost

35 0.684140	10.0.8.1	224.0.0.5	OSPF	84 Hello Packet
36 0.684542	10.0.8.1	224.0.0.5	OSPF	84 Hello Packet
37 0.831531	10.0.5.1	224.0.0.5	OSPF	124 LS Update
38 0.832279	10.0.5.1	224.0.0.5	OSPF	124 LS Update
39 0.833048	10.0.5.1	224.0.0.5	OSPF	124 LS Update
40 0.840927	10.0.3.2	224.0.0.5	OSPF	124 LS Update
41 0.844498	10.0.3.2	224.0.0.5	OSPF	124 LS Update
42 0.845123	10.0.3.2	224.0.0.5	OSPF	124 LS Update
43 0.860857	10.0.4.1	224.0.0.5	OSPF	124 LS Update
44 0.861216	10.0.4.1	224.0.0.5	OSPF	124 LS Update
45 0.861402	10.0.7.2	224.0.0.5	OSPF	124 LS Update
46 0.861728	10.0.7.2	224.0.0.5	OSPF	124 LS Update
47 0.861777	10.0.4.1	224.0.0.5	OSPF	124 LS Update
48 0.863027	10.0.7.2	224.0.0.5	OSPF	124 LS Update
49 0.863523	10.0.8.1	224.0.0.5	OSPF	124 LS Update
50 0.863837	10.0.8.1	224.0.0.5	OSPF	124 LS Update

Figure 28 n2 First Update Packet

87 1.197242	10.0.9.2	224.0.0.5	OSPF	80 LS Acknowledge
88 1.197506	10.0.9.2	224.0.0.5	OSPF	80 LS Acknowledge
89 1.197864	10.0.9.2	224.0.0.5	OSPF	80 LS Acknowledge
90 1.200061	10.0.7.1	224.0.0.5	OSPF	80 LS Acknowledge
91 1.200313	10.0.7.1	224.0.0.5	OSPF	80 LS Acknowledge
92 1.200699	10.0.7.1	224.0.0.5	OSPF	80 LS Acknowledge
93 1.201440	10.0.10.1	224.0.0.5	OSPF	80 LS Acknowledge
94 1.201688	10.0.10.1	224.0.0.5	OSPF	80 LS Acknowledge
95 1.202074	10.0.10.1	224.0.0.5	OSPF	80 LS Acknowledge

Figure 29 n2 Last ACK Packet

Comment on your findings:

From the timestamps in Figures 28 and 29, the first LS Update (frame 37) appears at about 0.831531 seconds, and the last LS Acknowledgment (frame 95) at about 1.202 seconds.

the time taken by the Network to exchange link state packets and adjust routing tables → Time taken = 1.202074 – 0.831531 = 0.371 sec

2.2.3. After execution of step 4, identify the new routing table and router database at router n2. Explain the updates in the new routing table and the new database.

```
Hello, this is Quagga (version 0.99.20.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

n2# show ip ospf database

       OSPF Router with ID (10.0.1.2)

                Router Link States (Area 0.0.0.0)

Link ID         ADV Router      Age  Seq#       CkSum  Link count
10.0.0.1        10.0.0.1        780 0x8000000b 0xf0be 3
10.0.0.2        10.0.0.2        820 0x8000000b 0x9809 3
10.0.1.2        10.0.1.2        819 0x8000000b 0x8b0b 3
10.0.3.1        10.0.3.1        139 0x80000012 0xb243 1
10.0.3.2        10.0.3.2        111 0x8000000c 0xf2a7 3
10.0.5.1        10.0.5.1        102 0x8000000c 0xfbaa 3
10.0.6.2        10.0.6.2        859 0x8000000a 0xcaac 3
10.0.8.2        10.0.8.2         98 0x8000000d 0x2823 4

                Net Link States (Area 0.0.0.0)

Link ID         ADV Router      Age  Seq#       CkSum
10.0.0.2        10.0.0.2        780 0x80000002 0x5dcc
10.0.1.2        10.0.1.2         88 0x80000002 0x62c3
10.0.2.1        10.0.1.2        779 0x80000002 0x4fd7
10.0.5.1        10.0.5.1       1422 0x80000002 0x67b2
10.0.6.2        10.0.6.2        170 0x80000002 0x49cd
10.0.7.1        10.0.6.2       2622 0x80000001 0x59bb
10.0.9.1        10.0.8.2       1049 0x80000002 0x2be4
10.0.10.2       10.0.8.2        748 0x80000002 0x5fa9

n2#
```

Figure 30 n2 Database after

```
n2# show ip ospf route
============ OSPF network routing table ============
N    10.0.0.0/24           [20] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.2.2, eth1
N    10.0.1.0/24           [10] area: 0.0.0.0
                           directly attached to eth0
N    10.0.2.0/24           [10] area: 0.0.0.0
                           directly attached to eth1
N    10.0.3.0/24           [40] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.9.1, eth2
N    10.0.4.0/24           [50] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.9.1, eth2
N    10.0.5.0/24           [40] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.9.1, eth2
N    10.0.6.0/24           [20] area: 0.0.0.0
                           via 10.0.1.1, eth0
N    10.0.7.0/24           [30] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.9.1, eth2
N    10.0.8.0/24           [20] area: 0.0.0.0
                           via 10.0.9.1, eth2
N    10.0.9.0/24           [10] area: 0.0.0.0
                           directly attached to eth2
N    10.0.10.0/24          [20] area: 0.0.0.0
                           via 10.0.9.1, eth2
N    10.0.11.0/24          [20] area: 0.0.0.0
                           via 10.0.2.2, eth1
N    10.0.12.0/24          [50] area: 0.0.0.0
                           via 10.0.1.1, eth0
                           via 10.0.9.1, eth2
N    10.0.13.0/24          [20] area: 0.0.0.0
                           via 10.0.9.1, eth2

============ OSPF router routing table =============
```

Figure 31 n2 Routing table after

Comment on your findings:

When router n4 goes offline, OSPF deletes every entry that mentioned n4. All routers quickly recalculate the best paths. Traffic that once used n4 now takes the next-best route, usually through n2, so the network stays up. The new route is a bit longer, so its cost number goes up, but nothing is unreachable.